



Article

Hierarchical Runtime Anomaly Diagnosis from Real PX4 Telemetry with Commit-Level Architecture-Aware Inspection Evidence

Jiao Wu ^{1,2,†}, Bo Liu ^{1,†}, Yilan Li ³, Chenglie Du ¹ and Junsheng Wu ^{1,*}

¹ Northwestern Polytechnical University, Xi'an 710072, Shaanxi, China; wjiao@mail.nwpu.edu.cn

² Xi'an Aeronautics Computing Technique Research Institute, Aviation Industry Corporation of China, Ltd., Xi'an 710065, Shaanxi, China;

³ Xi'an University of Technology, Xi'an, Shaanxi, China

* Correspondence: wujunsheng@mail.nwpu.edu.cn

† These authors contributed equally to this work.

Highlights

- Conditional four-domain classification is evaluated under chronological and log-isolated protocols.
- Traceable TreeSHAP aggregation is tested by conservation and attribution-guided masking.
- Commit-matched, Stage-1-gated analysis exposes detector losses before module inspection.
- Common-support paired replay reduces normal alarms with detection and latency limits reported.

Abstract

Runtime anomaly detection is important for improving the safety and reliability of unmanned aerial vehicles (UAVs), but identifying abnormal telemetry alone provides limited support for subsequent diagnosis and software inspection. Engineers further need to determine the affected fault domain, identify the telemetry evidence associated with the abnormal behavior, and narrow the software components that warrant inspection. To address this problem, we propose HS-AeroTS, a hierarchical framework for UAV telemetry anomaly diagnosis and architecture-aware software inspection. HS-AeroTS first performs anomaly detection and conditional four-domain diagnosis using temporal-statistical telemetry features. It then aggregates TreeSHAP contributions from statistical features to telemetry channels, uORB topics, and functional subsystems, and propagates topic-level evidence through a commit-specific PX4 uORB dependency graph to generate software-module inspection candidates. Experiments on the public UAV-SEAD benchmark, comprising 1,389 usable flight logs, show that the four-domain classifier achieves a Macro-F1 of 0.904 under leave-log-out evaluation. Attribution-guided masking further demonstrates the relevance of the identified telemetry evidence. In controlled PX4 replay experiments, common-support paired processing reduces normal alarms from 11/12 to 3/12 while retaining 10/12 fault detections. These results demonstrate the effectiveness of HS-AeroTS in connecting real-flight anomaly diagnosis with traceable, architecture-aware software inspection evidence.

Received:

Accepted:

Published:

Copyright: © 2026 by the authors.

Submitted to *Drones* for possible open access publication under the terms and conditions of the [Creative Commons Attribution \(CC BY\) license](#).

Keywords: UAV telemetry; runtime anomaly detection; fault-domain diagnosis; explainable machine learning; PX4; uORB; source-tree inspection candidates

1. Introduction

Unmanned aerial vehicles (UAVs) are increasingly used in environmental monitoring, infrastructure inspection, emergency response, and other autonomous applications. As UAV systems become more complex, ensuring flight safety and reliability has become increasingly important. During operation, the flight-control system continuously records heterogeneous telemetry, including position estimates, inertial measurements, actuator responses, estimator innovations, battery states, and system-health variables. These signals provide valuable information about the runtime state of the vehicle and have therefore been widely used for UAV anomaly detection and fault diagnosis [1–6,15].

Recent data-driven methods have shown strong capability in identifying abnormal flight states from multivariate telemetry. However, detecting an anomaly is only the first step toward practical diagnosis. Once abnormal behavior is observed, engineers still need to determine which fault domain is involved, which telemetry signals provide the strongest diagnostic evidence, and which software components should be inspected. Existing studies usually address these tasks separately: anomaly-detection methods focus on identifying abnormal states, explainable machine-learning methods provide feature-level attribution, and software fault-localization techniques operate on software-level evidence. The connection between telemetry-level diagnosis and flight-control software architecture therefore remains insufficiently explored [3,5,8,10].

UAV telemetry provides several properties that make such a connection feasible. Temporal-statistical features can summarize local variations in multivariate flight signals into compact and interpretable representations, while tree-based models are well suited to learning from these structured features [25,26]. Their predictions can be further explained using TreeSHAP, whose additive attributions can be aggregated from statistical features to physical telemetry channels and uORB topics [8,9]. In PX4, uORB publisher–subscriber relationships explicitly connect telemetry topics with software modules, providing a natural architectural bridge from model evidence to software inspection [11,12].

To exploit these properties, we propose HS-AeroTS, a hierarchical framework for UAV telemetry anomaly diagnosis and architecture-aware software inspection, as illustrated in Figure 1. HS-AeroTS first performs runtime anomaly detection and conditional diagnosis over four fault domains: External Position, Global Position, Altitude, and Mechanical/Electrical. It then aggregates TreeSHAP contributions from temporal-statistical features to telemetry channels, uORB topics, and functional subsystems. Finally, the resulting topic-level evidence is mapped through a firmware-specific PX4 uORB dependency graph to generate software-module inspection candidates. This design connects anomaly detection, fault-domain diagnosis, interpretable telemetry evidence, and software architecture within a unified diagnostic pipeline.

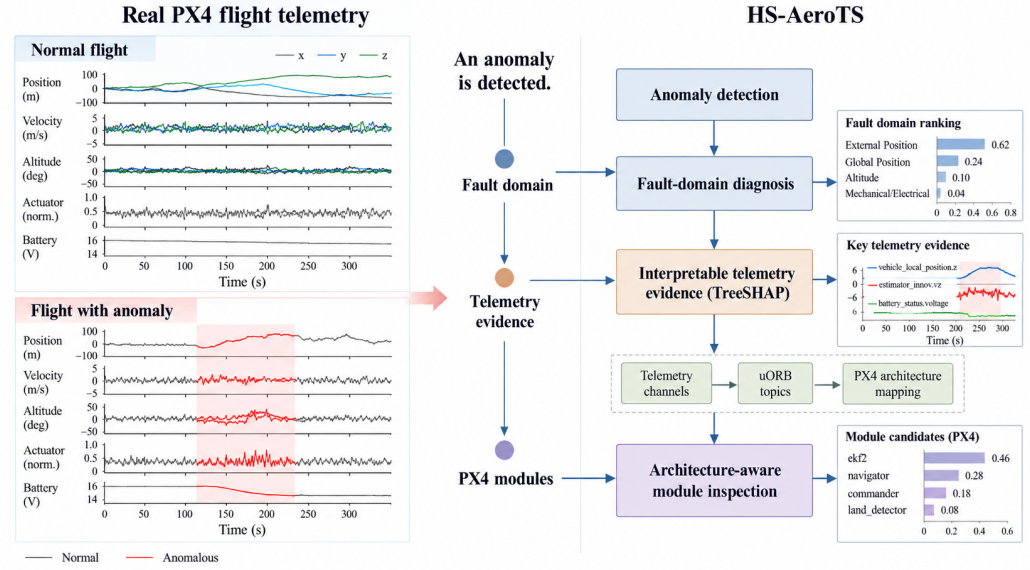


Figure 1. Overview of HS-AeroTS. Real PX4 flight telemetry is processed through runtime anomaly detection and fault-domain diagnosis. TreeSHAP contributions are aggregated to telemetry channels, uORB topics, and functional subsystems, after which firmware-specific PX4 uORB relations produce architecture-aware software-module inspection candidates.

We evaluate HS-AeroTS on the public UAV-SEAD benchmark using both chronological and flight-isolated settings [6]. The diagnostic evidence is further examined through attribution-based analyses and controlled PX4 replay experiments to assess its relevance and architecture-aware module ranking capability.

Our contributions are as follows:

- We propose HS-AeroTS, a hierarchical framework that extends UAV telemetry anomaly detection to fault-domain diagnosis and interpretable telemetry evidence.
- We develop a traceable evidence chain that aggregates TreeSHAP contributions from temporal-statistical features to telemetry channels and uORB topics, and further relates them to PX4 software-module inspection candidates through firmware-specific uORB relations.
- We evaluate the framework on the public UAV-SEAD benchmark and controlled PX4 replay, covering flight-level generalization, telemetry-evidence analysis, and architecture-aware software inspection.

2. Related Work

2.1. UAV Telemetry Anomaly Detection and Fault Diagnosis

UAV anomaly detection and fault diagnosis aim to identify abnormal operating states from flight measurements and determine the corresponding fault conditions. Existing approaches can be broadly categorized as knowledge-based, model-based, signal-based, and data-driven methods [1,2,15]. With the increasing availability of onboard flight logs, data-driven approaches have received growing attention because multivariate telemetry records the coupled behavior of state estimation, inertial sensing, actuation, power systems, and other flight-control components. Real-flight datasets such as ALFA and UAV-SEAD further provide annotated telemetry for evaluating anomaly detection and fault diagnosis under realistic operating conditions [4,6,16].

Early data-driven studies mainly relied on conventional machine-learning techniques to identify deviations from normal flight behavior. More recent methods increasingly exploit temporal and cross-channel dependencies using reconstruction models, probabilistic

sequence models, graph neural networks, and attention-based architectures [18–22]. In UAV-specific anomaly detection, MTCL-UAV employs adaptive multiscale Transformer modeling and contrastive learning to capture temporal variations and correlations in flight data [5], while LDC-P-VAE introduces dynamics-constrained probabilistic modeling for adaptive anomaly detection [7]. These methods demonstrate the effectiveness of learning richer temporal representations from UAV telemetry and have substantially advanced flight-data anomaly detection.

Despite this progress, most existing methods primarily produce anomaly scores, binary alarms, or predefined fault labels. Such outputs are effective for determining whether abnormal behavior occurs, but provide limited support for subsequent engineering inspection. In particular, the model evidence behind a fault prediction is rarely organized into physical telemetry channels or connected further to the flight-control software architecture. Our work builds on UAV telemetry anomaly detection and fault diagnosis by extending the diagnostic process from abnormal-state identification toward interpretable telemetry evidence and architecture-aware software inspection [8,9,11,12].

2.2. Explainable Models for Time-Series Diagnosis

Explainable machine-learning methods aim to reveal which input variables contribute to a model prediction. Representative post-hoc approaches include LIME, Integrated Gradients, and SHAP, which explain predictions through local surrogate models, accumulated gradients, or additive feature contributions, respectively [8,32,33]. For sequential data, TimeSHAP extends Shapley-based attribution to recurrent time-series models and can quantify the relevance of features, events, and time steps [34]. These methods have improved the interpretability of otherwise opaque predictive models and provide useful tools for analyzing multivariate time-series decisions.

Among these approaches, SHAP is particularly suitable for structured tabular representations because its additive attributions can be associated with individual input features. For tree ensembles, TreeSHAP provides an efficient formulation for computing such contributions and supports both local and global interpretation [8,9]. This property is useful for UAV telemetry, where a flight window is represented by multiple statistical descriptors derived from physical sensor and state variables.

However, most existing explainability methods operate at the level of the model input representation. In UAV telemetry analysis, an attribution assigned to a statistical descriptor does not directly indicate which physical telemetry channel, uORB topic, or functional subsystem is most relevant to the diagnosed fault. Bridging this gap requires organizing feature-level attributions into engineering-level evidence that can be traced from model inputs to the underlying telemetry structure. HS-AeroTS builds on TreeSHAP in this direction by aggregating temporal-statistical contributions into progressively higher-level telemetry representations for subsequent architecture-aware diagnosis.

2.3. PX4 Architecture and Software Fault Localization

PX4 adopts a modular flight-control architecture in which estimators, controllers, sensor drivers, and other software components exchange data through the uORB publish–subscribe middleware [11,12]. Many of these uORB topics are also recorded in ULog flight telemetry, providing an explicit relationship between runtime flight data and the software modules that produce or consume the corresponding messages. This architecture makes uORB a useful interface for relating telemetry-level observations to the internal structure of the flight-control software.

Software fault localization aims to reduce debugging effort by ranking program entities according to their likelihood of being associated with a failure. Traditional approaches

commonly exploit execution spectra, test outcomes, or other program-level evidence to rank suspicious statements, methods, or files [10,38]. Subsequent studies have also investigated the use of seeded faults and real defects to evaluate localization effectiveness under controlled ground truth [13,14]. These techniques provide useful principles for narrowing the software inspection space, but they generally assume direct software-level execution or testing evidence.

In telemetry-driven UAV diagnosis, such software-level evidence is usually unavailable during real flight. Instead, the available observations consist of abnormal telemetry signals and model-derived diagnostic evidence. The correspondence between uORB topics and PX4 modules therefore provides a natural bridge between telemetry analysis and software inspection. However, this connection has received limited attention in existing UAV anomaly-diagnosis studies. HS-AeroTS builds on this architectural relationship by linking topic-level telemetry evidence with firmware-specific PX4 module relations to support software-module inspection.

3. Materials and Methods

3.1. Problem Formulation

Let the recorded-flight corpus be $\mathcal{D} = \{(\mathbf{X}^{(n)}, \mathcal{A}^{(n)}, v_n)\}_{n=1}^{N_{\log}}$, where $\mathbf{X}^{(n)} \in \mathbb{R}^{T_n \times C}$ is the aligned multivariate telemetry of flight log n , $\mathcal{A}^{(n)}$ is its set of annotated anomaly intervals and fault types, and v_n is the PX4 firmware revision stored in the ULog metadata. A flight-aware windowing operator transforms each log into windows $w \in \mathcal{W}$ without crossing log boundaries. Each window is represented by a temporal–statistical feature vector $\mathbf{f}_w \in \mathbb{R}^D$ and retains its source-log identifier $g(w)$. In this study, $C = 87$ channels and $D = 1566$ features; the alignment, labeling, and feature construction are specified in Section 3.3.

Each window has a binary runtime label $y_w^{\text{bin}} \in \{0, 1\}$, where 1 denotes overlap with an annotated anomalous interval under the stated prediction horizon. When an explicit anomaly type is available, it also has a fault-domain label $y_w^{\text{dom}} \in \mathcal{Y}_{\text{dom}}$, with

$$\mathcal{Y}_{\text{dom}} = \{\text{External Position, Global Position, Altitude, Mechanical/Electrical}\}.$$

Normal and Uncategorized windows do not belong to this conditional four-domain label space. The predictive problem therefore comprises a binary detector $h_1 : \mathbb{R}^D \rightarrow [0, 1]$ and a conditional domain classifier $h_2 : \mathbb{R}^D \rightarrow \Delta^4$, where Δ^4 denotes the four-class probability simplex. With anomaly score $p_1(\mathbf{f}_w) = h_1(\mathbf{f}_w)$, domain probabilities $\mathbf{p}_2(\mathbf{f}_w) = h_2(\mathbf{f}_w)$, and a validation-selected threshold τ , the complete Cascade prediction is

$$\hat{y}_w^{\text{cas}} = \begin{cases} \text{Normal}, & p_1(\mathbf{f}_w) \leq \tau, \\ \arg \max_{d \in \mathcal{Y}_{\text{dom}}} p_{2,d}(\mathbf{f}_w), & p_1(\mathbf{f}_w) > \tau. \end{cases} \quad (1)$$

Thus, binary detection, ground-truth-anomaly-conditioned diagnosis, and complete five-class prediction are distinct evaluation tasks. In particular, performance of h_2 on labelled anomalous windows is not an end-to-end estimate of Equation (1).

For diagnostic evidence, let $\boldsymbol{\phi}_w^{(d)} \in \mathbb{R}^D$ be the TreeSHAP contribution vector for the predicted fault domain $\hat{d}$. Fixed provenance maps aggregate its absolute contributions through feature, telemetry-channel, uORB-topic, and functional-subsystem levels. For logs whose firmware matches revision v , a static uORB graph $G_v = (\mathcal{T}, \mathcal{M}, \mathcal{E}_v)$ relates topics $\mathcal{T}$ to source-tree modules $\mathcal{M}$. A propagation rule converts topic evidence into module scores $S_{w,m}$ and a ranked inspection list. This second problem is to preserve and test the model

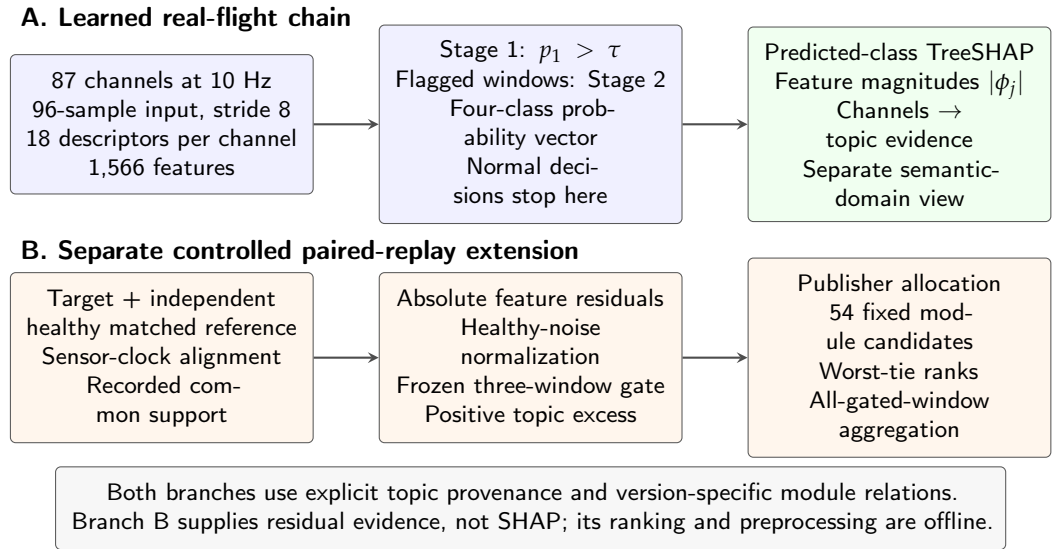


Figure 2. Method interfaces. (A) The learned real-flight chain uses Stage 1 screening, conditional Stage 2 diagnosis, and predicted-class SHAP evidence. Channels have separate topic and semantic-domain aggregations; topic evidence supplies the static graph. (B) The separately evaluated paired-replay extension uses independent healthy references and residual evidence with fixed publisher allocation. Ground-truth-anomaly-conditioned analyses of branch A are distinguished from its deployment-style gate. Neither branch validates causal module localization or onboard real-time operation.

evidence while reducing the source-tree inspection space; it is not to infer a causal defective module from real-flight labels.

Accordingly, the study evaluates three linked but non-interchangeable outputs: (i) runtime detection and fault-domain predictions against real-flight labels, (ii) hierarchical attribution evidence through conservation, masking, and semantic-map agreement, and (iii) module rankings against controlled source-mutation ground truth. The third output is evaluated using corrected controlled replay and a separately defined matched-reference residual extension. Onset-conditioned historical runs with inadequate input coverage are retained only as an execution audit, not capability estimates. The following subsection presents the overall framework implementing these definitions.

3.2. Overall Framework

HS-AeroTS implements the tasks defined in Section 3.1 and produces two related outputs: a runtime anomaly or normal decision with a conditional fault-domain prediction, and a traceable set of telemetry- and architecture-aware diagnostic evidence. As shown in Figure 2, the method first aligns heterogeneous uORB-derived channels, constructs flight-aware windows, and converts each window into temporal-statistical features. Stage 1 performs binary runtime anomaly detection, and Stage 2 performs four-class fault-domain diagnosis for anomalous windows. For a Stage 2 prediction, TreeSHAP contributions are aggregated from statistical features to telemetry channels, PX4 uORB topics, and functional subsystems. For firmware-matching data, topic evidence is then propagated through a commit-specific static uORB graph to produce software-module suspects for engineering inspection. The data-processing, diagnosis, explanation, and architecture-aware layers are described below in this order; each layer has its own input, output, and evaluation boundary.

3.3. Data Preprocessing and Feature Representation

UAV-SEAD was used as the primary real-flight dataset [6]. The input to this layer is a collection of PX4 ULog files and their anomaly annotations. To make the data inventory

reproducible, the dataset revision was fixed to 774a336a1d1d06b248927a35c2ea1f6869e9e1a4, and the original ULog files were retained as read-only inputs. Inventory and parsing checks excluded one missing file, two parse failures, and four excessive-duration or corrupt logs, leaving 1,389 usable logs. Numeric fields were first restricted to the state-estimation-related topic-field candidates defined in the preprocessing code, and fields with fewer than two finite timestamped samples or no measurable variation were excluded. A channel was then retained when it occurred in at least 60% of the usable logs, producing a common set of 87 telemetry channels. The output of this step is a flight-indexed set of raw channel sequences and the associated annotation intervals; aligned arrays and feature matrices were generated as derived artifacts by versioned scripts and configuration files.

Because uORB topics are logged asynchronously, each flight was resampled independently onto a uniform 10 Hz grid with a 100,000- μ s step. Let $x_{c,t}$ denote the value of retained channel c at aligned time index t . Finite samples within a channel were linearly interpolated by timestamp, values outside the observed interval were extended from the nearest finite endpoint, and a channel absent from an entire log was initialized to zero. Flight boundaries were never crossed during interpolation. This procedure yielded 1,892,063 aligned records across the 1,389 logs. The log identifier and the original annotation intervals were retained alongside every aligned sequence, so window construction and labeling never crossed a log boundary. This does not imply that every evaluation partition contains disjoint source logs: the fixed chronological reproduction can split a log across partitions, whereas leave-log-out assigns entire logs to one partition.

Each aligned flight sequence was converted into overlapping input windows. For window index w , $\mathbf{x}_{w,c}$ denotes the $L = 96$ aligned samples of channel c , and consecutive windows start $s = 8$ samples apart. A horizon of $h = 12$ additional samples was used only for label assignment and was not included in $\mathbf{x}_{w,c}$. The binary label y_w^{bin} was set to one if an annotated anomaly occurred anywhere from the window start through the end of this horizon. The window type was selected by the majority nonzero anomaly-type code over the same window-plus-horizon segment; codes 1–4 denote the four explicit fault domains and code 5 denotes Uncategorized. Thus, y_w^{bin} defines the Stage 1 target, whereas Stage 2 retains only windows whose majority type is one of the four explicit domains. Under the primary chronological protocol, this construction produced 218,537 windows. Split assignment preceded normalization. For channel c , the mean μ_c^{tr} and standard deviation σ_c^{tr} were estimated only from samples participating in training windows, after which all partitions were transformed as

$$z_{c,t} = \frac{x_{c,t} - \mu_c^{\text{tr}}}{\tilde{\sigma}_c^{\text{tr}}}, \quad \tilde{\sigma}_c^{\text{tr}} = \begin{cases} \sigma_c^{\text{tr}}, & \sigma_c^{\text{tr}} \geq 10^{-6}, \\ 1, & \sigma_c^{\text{tr}} < 10^{-6}. \end{cases} \quad (2)$$

Here, $c \in \{1, \dots, 87\}$ indexes the retained channels, t indexes the aligned samples, and $\tilde{\sigma}_c^{\text{tr}}$ prevents division by a near-zero training standard deviation. Within each protocol, this ordering restricts estimation of the feature scale to training-window samples. The fixed 87-channel dictionary, however, was selected using unlabelled coverage across the corpus; it is not a training-only channel-discovery experiment. The fixed chronological, purged, and leave-log-out assignments are detailed in Table 1.

Following the temporal-statistical representation used by AeroTSBoost [26], each normalized channel window was summarized by 18 descriptors: mean, standard deviation, minimum, maximum, range, five quantiles (10th, 25th, 50th, 75th, and 90th), start value, end value, endpoint difference, mean and standard deviation of first differences, mean and maximum absolute first difference, and lag-one autocorrelation. Let $d_k(\mathbf{z}_{w,c})$ denote

descriptor k applied to the normalized window of channel c , where $k \in \{1, \dots, 18\}$. For window w , concatenation over the 87 channels gives

$$\mathbf{f}_w = [d_k(\mathbf{z}_{w,c})]_{k=1,\dots,18;c=1,\dots,87} \in \mathbb{R}^{1566}. \quad (3)$$

The vector therefore contains $18 \times 87 = 1566$ features. If j indexes a coordinate of $\mathbf{f}_w$, the accompanying feature dictionary stores its descriptor index k , telemetry channel c , and PX4 topic. This explicit provenance is the interface between preprocessing and the explanation layer: it allows each feature contribution to be assigned to a channel and then to a topic and subsystem without reconstructing names from strings.

3.4. Hierarchical Anomaly Detection and Fault-Domain Diagnosis

HS-AeroTS separates runtime anomaly detection from conditional fault-domain diagnosis according to Equation (1). Its input is the 1566-dimensional feature vector $\mathbf{f}_w$ defined in Section 3.3, and its outputs are a Stage 1 anomaly score, a Stage 2 fault-domain probability vector when diagnosis is performed, and a final runtime class under the Cascade. Stage 1 is a class-balanced binary LightGBM model configured with 1600 estimators, learning rate 0.025, 64 leaves, minimum child size 80, subsample 0.90, feature subsampling 0.75, ℓ_1 regularization 0.20, ℓ_2 regularization 8.0, and 80-round early stopping. It produces an anomaly score $p_1(\mathbf{f}_w) \in [0, 1]$ for each feature window. Its operating threshold τ is selected by the implemented precision–recall search on validation labels, using the threshold with maximum F1, and is then frozen for test evaluation. AUPRC and AUROC assess threshold-independent detection, while threshold-dependent F1 and event-level measures assess the operating point.

Stage 2 is a four-class LightGBM model configured with 1200 estimators, learning rate 0.025, 64 leaves, minimum child size 30, subsample 0.90, feature subsampling 0.75, ℓ_1 regularization 0.20, ℓ_2 regularization 8.0, class-balanced weighting, and 80-round early stopping. It takes the same feature vector and is trained only on windows with an explicit anomalous fault-domain label $y_w^{\text{dom}} \in \{1, 2, 3, 4\}$. These codes denote External Position, Global Position, Altitude, and Mechanical/Electrical, respectively. Normal windows have code 0 and Uncategorized windows have code 5; both are excluded from this conditional training task. Its output is a four-class probability vector $\mathbf{p}_2(\mathbf{f}_w)$ and a predicted domain $\hat{y}_w^{\text{dom}} = \arg \max_d p_{2,d}(\mathbf{f}_w)$. Stage 2 uses the same log membership as Stage 1 and is evaluated with Macro-F1, Balanced Accuracy, Macro Recall, and per-class metrics. Core model parameters are reported in Table 1, while frozen machine-readable configurations and run records are part of the analysis package described in the Data Availability Statement.

Thus, Stage 1 is a binary screening task, Stage 2 is a conditional four-class diagnosis task, and Cascade is the complete five-class runtime decision formed by gating Stage 2 with Stage 1. Direct Five-Class is the comparator that predicts Normal plus the four fault domains in one five-class LightGBM model without this gate. Random Forest uses the configured 500 trees, square-root feature subsampling, minimum leaf size 2, and balanced subsampling; majority and stratified-random methods are used as reference methods under the corresponding task definitions. The distinction is important for interpreting results: conditional Stage 2 metrics are not complete five-class Cascade metrics, and the Cascade is not assumed to outperform Direct Five-Class under every protocol.

3.5. SHAP-Based Explainable Evidence

For each Stage 2 input $\mathbf{f}_i$, the LightGBM native contribution interface returns one contribution vector for each class. The class is selected by the maximum predicted probability, and the contribution associated with that predicted class is retained; the final expected-value term returned by LightGBM is excluded from feature aggregation. Let $i \in \{1, \dots, N\}$

index an evaluated anomalous window, $j \in \{1, \dots, 1566\}$ index a temporal-statistical feature, and $\hat{c}_i$ denote the Stage 2 predicted fault domain. If $\phi_{ij}^{(\hat{c}_i)}$ is the TreeSHAP contribution of feature j to the raw score for class $\hat{c}_i$, then

$$e_{ij} = \left| \phi_{ij}^{(\hat{c}_i)} \right|.$$

The feature dictionary and the declared semantic mapping define the aggregation sets. Let $q \in \{1, \dots, Q\}$ index telemetry channels, t index PX4 uORB topics, and s index functional subsystems. Let $\mathcal{F}(q)$ be the set of feature indices belonging to channel q , $\mathcal{Q}(t)$ the set of channels mapped to topic t , and $\mathcal{Q}(s)$ the set of channels assigned to subsystem s . Channel-, topic-, and subsystem-level evidence is obtained by summing feature evidence through the hierarchy:

$$E_{iq} = \sum_{j \in \mathcal{F}(q)} e_{ij}, \quad E_{it} = \sum_{q \in \mathcal{Q}(t)} E_{iq}, \quad E_{is} = \sum_{q \in \mathcal{Q}(s)} E_{iq}.$$

Here, the first sum groups the 18 descriptor contributions belonging to a channel, the second maps channels to their logged uORB topics, and the third maps channels to the declared functional subsystems. The sets are applied to evidence for the same window i ; no evidence from different windows is mixed during local aggregation.

Aggregation conservation is checked at each level. For an aggregation level L with units $\mathcal{U}_L$ and unit evidence $E_{iu}^{(L)}$, define

$$\epsilon_{i,L} = \sum_{u \in \mathcal{U}_L} E_{iu}^{(L)} - \sum_{j=1}^{1566} e_{ij}, \quad L \in \{\text{channel, topic, subsystem}\}.$$

For example, $E_{iq}^{(\text{channel})} = E_{iq}$, $E_{it}^{(\text{topic})} = E_{it}$, and $E_{is}^{(\text{subsystem})} = E_{is}$. A value close to zero indicates numerical conservation under the specified mapping; it does not indicate physical causality.

Evidence quality is evaluated by validation-derived SHAP top- k masking against equal-size random masking, and by channel-level agreement with a document-derived semantic channel-to-subsystem mapping. Let T_i^K be the set of K highest-evidence channels for window i , $g(q)$ the declared subsystem of channel q , and y_i the fault-domain label. Let $\mathbb{I}[\cdot]$ be the indicator function. We report

$$C_i(K) = \frac{1}{K} \sum_{q \in T_i^K} \mathbb{I}[g(q) = y_i], \quad \text{Consistency@K} = \frac{1}{N} \sum_{i=1}^N C_i(K),$$

$$\text{Hit@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left[\sum_{q \in T_i^K} \mathbb{I}[g(q) = y_i] > 0 \right].$$

For non-negative channel evidence E_{iq} , weighted consistency is

$$\text{WeightedConsistency} = \frac{1}{N} \sum_{i=1}^N \frac{\sum_{q=1}^Q E_{iq} \mathbb{I}[g(q) = y_i]}{\sum_{q=1}^Q E_{iq}}.$$

If $Q = 87$ and n_y channels are assigned to domain y , the exact without-replacement random baselines are n_y/Q for Consistency@K and $1 - \binom{Q-n_y}{K} / \binom{Q}{K}$ for Hit@K, averaged over the evaluated labels. The reference mapping is defined from PX4 message topics and field semantics without access to SHAP values. It corrects generic string-token leakage in the original rule, such as attitude yaw and manual-control axes being interpreted as global-position fields; 11 of 87 channel assignments change. Channels assigned “Shared” count as

non-matches for every specific fault domain. Because the reference is author-defined rather than independently expert-adjudicated, all of these quantities are mapping sensitivities, not subsystem ground-truth validation. The complete 87-channel mapping is given in Appendix A.

The primary masking test ranks features by mean absolute predicted-class TreeSHAP on validation anomaly windows, then replaces selected test features with feature-wise medians computed from a deterministic sample of 20,000 training windows (seed 20260820). Equal-size random masks are repeated 20 times within each model seed. A sensitivity analysis instead replaces all selected features in each test window with their joint values from one deterministically sampled training-window donor. The donor preserves dependence within the replaced block, although combining it with unreplaced test features can still be off-manifold. Random feature subsets are nested within each model seed. Paired uncertainty is estimated by resampling complete flight logs and averaging the contrast across the five model seeds. These tests assess numerical organization and prediction sensitivity under specified perturbations; they do not establish physical causality or feature necessity.

3.6. PX4 Software-Module Suspect Ranking

The architecture-aware layer maps topic-level SHAP evidence to commit-specific PX4 software-module inspection candidates. Its input is the topic-level evidence E_{it} from Section 3.5 together with firmware provenance retained in each ULog header. The corpus contained seven firmware commits, and the most frequent commit, 82aa24adfc29321cfd1209e287eab6c2b16780e, occurred in 562 of 1389 usable logs; its source tree was selected for static analysis. All architecture-aware aggregate results are recomputed only from matching-commit logs. In the fixed test partition, this restriction retains 14,419 of 34,059 windows (42.3%) from 561 matching-commit logs and 1,393 of 3,602 ground-truth anomalous windows (38.7%). Windows from the other six commits are excluded rather than propagated through a mismatched graph. The output of the matching step is therefore a firmware-coherent subset of windows, not a prediction for logs from an unmatched revision.

For the selected commit, the source scanner searches the configured PX4 C/C++ source globs for `ORB_ID(topic)` occurrences, assigns publisher or subscriber roles using the nearest recognized uORB API hint within a 500-character local context, and converts source paths into module identifiers. These records define a static graph $G = (\mathcal{T}, \mathcal{M}, \mathcal{E})$, where $\mathcal{T}$ is the set of evaluated topics, $\mathcal{M}$ is the set of source-tree modules, and $\mathcal{E}$ contains topic–role–module records; the propagation mode r selects a subset or interpretation of this fixed edge set. An internal consistency check covers all 244 unambiguous same-statement declarations or calls involving the 18 evaluated topics. Within this explicit scope, agreement with the statement-level role rule and recovery of its explicit edges are both 1.000. This is not an independent gold audit because the reference and parser use the same uORB hint vocabulary at different context granularities; it does not quantify recall for macro-generated, indirect, conditionally compiled, or runtime-created relations.

The initial source-tree scan contains 180 edges and 54 modules. A conservative-exclusion sensitivity removes paths containing examples, templates, tests, POSIX, QURT, Bebop, Linux, Snapdragon, simulator, or replay markers for the log-confirmed PX4FMU_V2 and NuttX environment. The filtered edge list must be a strict subset of the original graph and is checked for evidence conservation. This rule is deliberately path-based: neither the unfiltered nor filtered graph is an actual target-specific compiled build graph. Full exclusion rules and software settings are listed in Appendix B. The graph is consequently a

static source-tree representation of topic connectivity, not a compiled or runtime execution graph.

For propagation mode r , let $\mathcal{M}_t^{(r)} \subseteq \mathcal{M}$ be the modules connected to topic t and E_{it} the topic-level evidence for window i . Evidence is allocated uniformly across the connected modules:

$$A_{it,m}^{(r)} = \begin{cases} E_{it}/|\mathcal{M}_t^{(r)}|, & m \in \mathcal{M}_t^{(r)}, \\ 0, & \text{otherwise,} \end{cases} \quad S_{i,m}^{(r)} = \sum_t A_{it,m}^{(r)}.$$

Here, $A_{it,m}^{(r)}$ is the share of topic t 's evidence assigned to module m for window i , and $S_{i,m}^{(r)}$ is the resulting module suspect score before ranking. The uniform allocation preserves each topic's evidence mass within a propagation mode, while the number of connected modules determines how concentrated or diluted that topic's contribution becomes.

The static SHAP analysis evaluates producer-only, consumer-only, and bidirectional propagation. A sensitivity divides each module's aggregate score by its number of mapped topics before renormalization, exposing dependence on graph degree. Scores can be retained per window or aggregated over the stated matching-commit set; this aggregation does not identify a known defective module in real flights. The paired-replay extension below reuses the publisher relations but supplies residual evidence instead of TreeSHAP evidence, and retains the full 54-module candidate universe. It does not apply the 36-module real-flight exclusion sensitivity to improve replay ranks. Mapping coverage, conservation, and real-flight propagation sensitivity are reported in Section 4.4.

3.7. Matched-Reference Residual Extension for Controlled Replay

The extension tests whether a target replay differs from an independent healthy replay of the same source log. It reuses the 87 channels, 18 descriptors per channel, frozen scaler, and publisher mapping, but replaces both the learned detection score and the downstream SHAP evidence. Neither target mutation type, injected onset, nor ground-truth module is an input to prediction. Distinct file hashes verify that the healthy target and its reference are different replay outputs. Sensor payload matches among the first 256 six-axis samples establish a constant offset to the source clock without using the injected effect. The target header clock is restored only for reporting detection times.

For feature j and window w , define the absolute paired difference $d_{wj} = |f_{wj}^{\text{target}} - f_{wj}^{\text{ref}}|$. In the original paired frontend, reference features are interpolated to target feature times only within the shared basic-sensor support. Independent healthy replay pairs from the other two development source flights estimate a feature noise scale

$$n_j = \max\{Q_{0.99}(d_{\cdot j}^{\text{healthy}}), 0.05\}, \quad r_{wt} = \max_{j \in \mathcal{F}(t)} d_{wj}/n_j, \quad s_w = \max_t r_{wt}, \quad (4)$$

where $Q_{0.99}$ is the implemented higher-order empirical quantile and $\mathcal{F}(t)$ contains the features of topic t . A gate opens at the third consecutive window with $s_w > \theta$, without backfilling earlier windows. Each development fold sets θ to the maximum healthy three-window sustained score, with a minimum of 3. Before transfer replay, the existing calibration with the largest numerical threshold, $\theta = 26.47158145904541$, and its associated noise vector were frozen. A larger threshold does not guarantee more conservative behavior across different noise vectors; all frozen alternatives are retained as sensitivities in the Supplementary Material.

For the gated set $\mathcal{G}$, topic evidence is

$$R_t = \frac{1}{|\mathcal{G}|} \sum_{w \in \mathcal{G}} \max(r_{wt} - \theta, 0), \quad S_m = \sum_{t: m \in \mathcal{M}_t^{(\text{producer})}} R_t / |\mathcal{M}_t^{(\text{producer})}|. \quad (5)$$

Both are zero if no window is gated. The original global gate permits different topics to dominate successive windows; same-channel persistence is a separate supplementary calibration variant. Equation (5) aggregates all gated windows, not an onset-defined interval, and yields an offline module ranking.

The common-support frontend changes the observation support before computing Equation (4). For each topic, exact shared source-clock timestamps are used if they contain at least two packets and cover at least half the shorter stream; otherwise each regenerated stream retains its timestamps. At 10 Hz, both streams must support every sample in a 96-sample channel window. There is no endpoint extrapolation, and interpolation gaps may not exceed the larger of 0.1 s and five times the channel-stream median sampling gap. Invalid channel windows contribute no residual evidence and are recorded as unavailable; fault trials remain in the denominator. Original descriptors, noise, threshold, and publisher allocation remain fixed for the principal frontend comparison. Because cadence and interpolation use complete recorded streams, this procedure is offline even though the three-window gate itself uses only present and past scores.

4. Experiments and Results

4.1. Experimental Setup and Protocols

The experiments address three questions: whether recorded PX4 telemetry supports runtime anomaly detection and conditional fault-domain diagnosis; whether hierarchical SHAP aggregation provides traceable, prediction-relevant diagnostic evidence; and whether commit-specific uORB relations can convert topic evidence into useful software-module suspect rankings under controlled source mutations. Table 1 summarizes the datasets, splits, evaluation roles, and permitted outputs. Detailed preprocessing and model definitions are given in Section 3 and are not repeated here; the measures used across all experiments are defined centrally in Section 4.2.

UAV-SEAD is the only dataset used to fit the supervised Stage 1 and Stage 2 models. The fixed chronological split provides the primary reproduction protocol, the 14-window purged split tests sensitivity to temporal proximity, and leave-log-out assigns complete flights to partitions to test unseen-log generalization. ALFA is used only for zero-shot binary detection because its labels are not mapped to the four UAV-SEAD fault domains. Uncategorized UAV-SEAD logs support only flight-level weak-label screening because they have no temporal anomaly ranges. Controlled PX4 replay is separate from real-flight model fitting and provides the only source-mutation ground truth for module-level ranking.

Baseline comparisons distinguish detector performance from the structure of diagnosis. Direct Five-Class is the principal structural comparator to the complete Cascade, while Random Forest, majority, and stratified-random references retain their corresponding supervised tasks. Resource-limited CATCH and linear VAR(1) feasibility results, together with their computational budgets and evaluation scope, are documented separately in Supplementary Material Section S6.

For the supervised real-flight models, thresholds and early stopping are selected on validation data only. Replay residual calibration instead uses development healthy pairs as specified below. Five model seeds are used for the principal real-flight experiments. Seed standard deviations describe fitted-model variation, whereas 95% cluster-bootstrap intervals describe sampling variation: complete flight logs are resampled for UAV-SEAD,

sequences for ALFA, and flights for Uncategorized screening. Absolute and paired real-flight analyses use 1000 repetitions with seed 20260825. New replay results are descriptive counts over three source flights per group; old intervals computed from incomplete replay inputs are not transferred to corrected results.

Corrected replay fixes PX4 commit 82aa24adfca29321cfd1209e287eab6c2b16780e and Ubuntu 20.04 WSL, without Gazebo, ROS, QGroundControl, or NuttX. The initial three development source logs and three additional transfer logs each have four mutation conditions (Commander, EKF2, INAV, and Land Detector), with a healthy reference, fault target, and independent normal target. Fixed mutation activation at 30 s is used for evaluation only. Source logs lack `ekf2_timestamps`, so EKF2 uses the generic publication adapter with `ekf2_start`, without the dedicated `-r` handshake. Input completion, clock alignment, finite estimator outputs, and the original effect checks are reported separately from predictive scores.

The first transfer protocol froze its residual parameters before evaluating the three additional sources. Those sources had appeared in the historical P2 scaler data, so the isolation concerns replay calibration, not a globally untouched preprocessing pipeline. Subsequent common-support and calibration analyses use the same already examined transfer sources and are exploratory. A two-reference sensitivity adds 24 healthy replays across the same six source logs, using the new runs as normal targets and the two older healthy outputs as references. These repeats add no independent source flights. Supplementary Material Sections S1 and S5 specify source identities, data roles, and all controls.

Table 1. Evaluation protocols and data partitions.

Protocol	Purpose	Data and evaluation set	Isolation / scope
Fixed chronological	Primary reproduction	1389 UAV-SEAD logs; 152,340/32,138/34,059 train/validation/test windows; five seeds	Chronological windows; overlapping source logs may span partitions
Purged	Temporal-proximity sensitivity	133,353/8,536/17,966 windows; 14-window boundary purge; five seeds	Removes overlap near split boundaries
Leave-log-out	Unseen-flight generalization	970/206/213 complete train/validation/test logs; 149,774/37,324/31,439 windows	No flight log crosses a partition
ALFA zero-shot	External binary transfer	46 sequences; 5273 windows; 16 shared channels	Frozen UAV-SEAD detector and validation threshold; no four-class transfer
Uncategorized	Weak-label screening	139 Uncategorized versus 898 pure-Normal logs	Flight-level metrics only; no temporal event labels
Paired PX4 replay	Controlled extension	3 development + 3 transfer sources; 12 fault targets per group; separate healthy targets/references	Frozen transfer followed by explicitly exploratory analyses; source identities in Supplement S1

All UAV-SEAD protocols use 96-sample windows, stride 8, horizon 12, 87 channels, and 1566 derived features. Sampling intervals use 1000 flight- or sequence-cluster bootstrap repetitions. Corrected replay counts are descriptive; source-flight dependence is retained. Frozen model configurations and per-run records are described in the Data Availability Statement.

4.2. Evaluation Metrics

Metrics are assigned to the three evidence levels defined in Section 3.1; a metric at one level is not used as evidence for another. For binary anomaly detection, area under the precision–recall curve (AUPRC) is the primary threshold-independent measure because anomalous windows are the minority class, while area under the receiver-operating-characteristic curve (AUROC) is reported as a complementary ranking measure. At the

operating point, the threshold τ is selected on validation data and frozen before test evaluation. Window-level precision P , recall R , and F1 are related by

$$F1 = \frac{2PR}{P + R}. \quad (6)$$

The independently optimized test F1 is reported only as an oracle descriptive quantity and is not treated as an operational estimate.

Log-aware event precision and recall are computed after separating windows by source flight. Within each log, consecutive positive windows form an event, and a predicted event is matched at most once to a ground-truth event when their index ranges overlap. Event F1 is then obtained from Equation (6). This grouping prevents events at adjacent log boundaries from being merged. Detection delay is defined only for a detected run and is measured from the annotated or injected onset to the first subsequent positive detector window; it is left undefined when the detector never activates.

For a K -class task, let P_k and R_k denote the precision and recall for class k . Macro-F1 and balanced accuracy are

$$\text{MacroF1} = \frac{1}{K} \sum_{k=1}^K \frac{2P_k R_k}{P_k + R_k}, \quad \text{BalancedAccuracy} = \frac{1}{K} \sum_{k=1}^K R_k. \quad (7)$$

These metrics give each class equal weight. They are reported separately for the conditional four-domain Stage 2 task and the complete five-class task; values across these two label spaces are not directly comparable. Per-class recall and F1 are additionally reported to expose class-specific failure modes.

Explanation quality is evaluated with complementary numerical, predictive, and semantic criteria. Conservation error measures the difference between total absolute feature evidence and its aggregate at each hierarchy level, as defined in Section 3.5. For a mask size k , the masking effect is

$$\Delta_m(k) = \text{MacroF1}_{\text{unmasked}} - \text{MacroF1}_{m,k}, \quad (8)$$

where m denotes SHAP-ranked or random masking; the paired advantage is $\Delta_{\text{SHAP}}(k) - \Delta_{\text{random}}(k)$. Consistency@ K , Hit@ K , and WeightedConsistency use the declared channel-to-subsystem map and the definitions in Section 3.5. They measure agreement with that semantic reference, not physical causality or independently adjudicated subsystem truth.

For controlled replay, let r_b be the rank of the known mutated module in run b , let M_b be the number of ranked modules, and let B be the number of evaluated runs. Module-ranking performance is summarized as

$$\text{Top-}K = \frac{1}{B} \sum_{b=1}^B \mathbb{I}[r_b \leq K], \quad \text{MRR} = \frac{1}{B} \sum_{b=1}^B \frac{1}{r_b}, \quad \text{EXAM} = \frac{1}{B} \sum_{b=1}^B \frac{r_b}{M_b}. \quad (9)$$

Higher Top- K and MRR indicate better candidate ranking. All fault trials remain in the denominator. Ranks use the worst position among tied module scores; a module with no positive evidence, or a run with no post-onset detection, contributes zero to Top- K and reciprocal rank. EXAM is defined for traceability but is not used to carry historical incomplete-run estimates into the revised replay results.

For a fault trial, detection recall counts whether any gated window occurs at or after its injected onset. The onset is read only during scoring, after predictions and rankings have been fixed. The normal-run false-alarm rate is the proportion of independent healthy targets with at least one alarm over the entire available run, not a window-level false-

positive rate. Pre-onset alarms are reported separately. A clean fault-specific detection additionally requires no alarm before the fault onset and no alarm in the corresponding normal target. This paired criterion is not deployment precision. Delay is measured to the first post-onset gated window only among detected trials, with misses reported separately; module evidence uses all gated windows and the delay is not localization-completion time. Observation duration and valid channel-window coverage accompany common-support results.

Across repeated model fits, tables report the mean and sample standard deviation over seeds. Sampling uncertainty is represented by 95% cluster-bootstrap confidence intervals, with complete flight logs as clusters for UAV-SEAD, sequences for ALFA, and flights for Uncategorized screening. Corrected replay summaries retain source-flight and mutation identities and do not claim precision from treating the 12 related trials as independent flights. Paired method contrasts are recomputed within each bootstrap replicate before aggregation; overlapping telemetry windows are never treated as independent bootstrap units.

4.3. Real-Telemetry Detection and Fault-Domain Diagnosis

This experiment asks whether recorded PX4 telemetry supports reproducible binary runtime anomaly detection, conditional four-domain diagnosis, and complete five-class prediction under the primary fixed chronological protocol. Dataset and split counts are summarized in Table 1; the results below distinguish the detector, anomaly-only Stage 2, and complete Cascade tasks.

Table 2 provides the principal fixed-chronological comparison. Results are grouped by prediction task because detection metrics, conditional-diagnosis metrics, and complete five-class metrics answer different questions and should not be compared across groups.

Table 2. Primary fixed-chronological test results grouped by prediction task.

Task	Method	AUPRC $\uparrow$	AUROC $\uparrow$	Macro-F1 $\uparrow$	Balanced accuracy $\uparrow$
Binary detection	HS-AeroTS Stage 1	0.7522 ± 0.0039	0.9512 ± 0.0006	—	—
Conditional four-domain diagnosis	HS-AeroTS Stage 2	—	—	0.8980 ± 0.0020	0.8863 ± 0.0021
	Random Forest	—	—	0.8724 ± 0.0037	0.8382 ± 0.0052
	Majority	—	—	0.1608	—
	Stratified random	—	—	0.2358 ± 0.0090	—
Complete five-class prediction	HS-AeroTS Cascade	—	—	0.6902 ± 0.0071	0.6755 ± 0.0152
	Direct Five-Class	—	—	0.6599 ± 0.0029	0.6144 ± 0.0021

Values are mean $\pm$ model-seed standard deviation over five seeds unless noted; “—” denotes a metric that is not applicable or was not reported. Budget-limited normal-only references are reported separately in Supplementary Material Section S6.

The five-seed LightGBM detector reproduced the reference performance level on the fixed chronological test partition. The mean AUROC was 0.9512 ± 0.0006 and the mean AUPRC was 0.7522 ± 0.0039 , with a 95% flight-log cluster-bootstrap CI of [0.6915, 0.8080]. The independently optimized test-set F1 was 0.6924 ± 0.0056 ; this is an oracle descriptive value and is not an operational estimate because its threshold uses test labels. These values were close to the reference values of 0.9516, 0.7516, and 0.6860, respectively, and preserved the reported detector performance trend. The AUPRC result is the primary threshold-independent measure because anomalous windows were a minority of the evaluated windows.

For operational use, the threshold was selected separately for each seed by maximizing validation F1 and then frozen before test evaluation. With this validation-derived threshold,

the test F1 was 0.6895 ± 0.0067 , and the log-aware event F1 was 0.6051 ± 0.0218 . Thus, the threshold-dependent operating result remained close to the independently optimized test F1, while the event-level measure was lower because it evaluates the temporal detection behavior at the frozen operating point.

The budget-limited CATCH and GCAD-inspired VAR(1) feasibility references use different training objectives and budgets from the supervised models. Their complete results are retained in Supplementary Material Section S6 and do not support a detector-superiority claim.

On the 3602 anomalous test windows, the anomaly-only Stage 2 LightGBM classifier attained Macro-F1 of 0.8980 ± 0.0020 (95% flight-log CI [0.8581, 0.9296]) and Balanced Accuracy of 0.8863 ± 0.0021 . Its mean per-class recall and F1 were 0.9685 and 0.9390 for External Position, 0.9279 and 0.9162 for Global Position, 0.7916 and 0.8503 for Altitude, and 0.8574 and 0.8865 for Mechanical/Electrical, respectively. The corresponding Random Forest baseline achieved Macro-F1 of 0.8724 ± 0.0037 and Balanced Accuracy of 0.8382 ± 0.0052 . The majority and stratified-random references achieved Macro-F1 values of 0.1608 and 0.2358 ± 0.0090 . These comparisons indicate that the four-class diagnostic task was learned under the specified ground-truth-anomaly-conditioned label protocol; they do not represent complete online five-class performance.

When Stage 1 thresholding and Stage 2 prediction were combined, the complete five-class Cascade achieved Macro-F1 of 0.6902 ± 0.0071 (95% flight-log CI [0.6308, 0.7350]) and Balanced Accuracy of 0.6755 ± 0.0152 . Its mean recall/F1 pairs were 0.9629/0.9632 for Normal, 0.7787/0.7088 for External Position, 0.6649/0.6493 for Global Position, 0.4329/0.5186 for Altitude, and 0.5379/0.6112 for Mechanical/Electrical. The lower anomaly-class performance relative to the conditional Stage 2 result reflects the additional Stage 1 gating step. The five-seed Direct Five-Class LightGBM comparator achieved Macro-F1 of 0.6599 ± 0.0029 (95% flight-log CI [0.5991, 0.7070]) and Balanced Accuracy of 0.6144 ± 0.0021 on the same fixed chronological test partition. These fixed-split values describe the primary reproduction protocol; stricter isolation results are reported separately.

4.4. SHAP Evidence and Software-Module Suspect Ranking

This experiment asks whether predicted-class TreeSHAP contributions can be organized into conserved and prediction-relevant telemetry evidence, and whether commit-specific uORB propagation provides useful software-module inspection candidates. The primary explanation analysis was ground-truth-anomaly-conditioned: it used all 3602 labelled anomalous test windows from the fixed chronological protocol and did not require Stage 1 to detect them. For each window, the TreeSHAP contributions of the predicted Stage 2 class were converted to absolute evidence and aggregated from statistical features to telemetry channels, PX4 topics, and subsystems. The aggregation preserved the total evidence mass: the maximum absolute conservation error across the hierarchy was 7.11×10^{-15} . This verifies only the numerical redistribution of evidence; it does not establish that the attributed channel or subsystem is the physical source of the anomaly.

The masking analysis tested whether the validation-derived SHAP ranking was more informative than an equal-size random feature selection. The top- k SHAP features were ranked on validation anomaly windows and replaced in the test data by feature-wise medians calculated from the training windows. The Macro-F1 decrease for SHAP masking increased from 0.0409 at $k = 10$ to 0.2683 at $k = 100$. Across 20 random subsets nested within each of five model seeds, the paired SHAP-minus-random advantage was 0.0404 (95% flight-log-bootstrap CI [0.0102, 0.0713]) at $k = 10$ and 0.2646 [0.2052, 0.3300] at $k = 100$. Replacing the selected feature block with values from sampled training rows produced drops of 0.0516 [0.0277, 0.0752] and 0.4955 [0.4604, 0.5283], respectively. The direction

therefore persisted under both median and joint-donor perturbations, although donor replacement can still create cross-block combinations with unreplaced features. These results support prediction-relevant feature concentration, not causal influence or physical necessity.

At the telemetry-channel level, agreement depends on the semantic reference. Under the document-derived mapping, overall Consistency@1 was 0.2500 ± 0.0056 , compared with the exact random baseline of 0.1707. External Position remained near random and Global Position below random; therefore, the positive masking result is evidence of prediction relevance, not uniform agreement with physical fault domains. Supplementary Material Section S2 reports the complete class-level results and the 11 changed channel assignments.

For the architecture-aware layer, the unfiltered static scan at commit 82aa24adfc29321cfd1209e287eab6c2b16780e contained 180 topic-role-module edges covering 54 PX4 modules. The 244-occurrence internal consistency check returned 1.000 agreement and edge recovery within its shared-vocabulary scope; it is not an independent uORB gold audit. Conservative source-path exclusion removed 35 edges and 18 modules, leaving 145 edges, 36 modules, and all 18 topics while preserving evidence conservation. Producer Top-3 overlap was 3/3 before versus after filtering, while Top-5 overlap was 4/5; bidirectional Top-5 was unchanged. Under filtered producer-only propagation, the highest-ranked source-tree inspection candidates were `src/modules/mavlink` (SHAP share 0.2572), `src/modules/ekf2` (0.2318), `src/modules/sensors` (0.1243), `src/modules/vtol_att_control` (0.0811), and `src/modules/position_estimator_inav` (0.0708). Figure 3(b) shows this conservative-exclusion result. The exclusion is path-based and does not reconstruct the compiled PX4FMU_V2 build; degree normalization and propagation-mode sensitivity further preclude interpreting the order as a stable root-cause ranking.

The separate Stage-1-gated matching-commit analysis used the frozen validation-derived detector thresholds. Averaged across the five seeds, the 14,419 matching-commit test windows comprised 937.2 true positives, 455.8 false negatives, 532.2 false positives, and 12,493.8 true negatives; mean detector recall was 0.6728 ± 0.0248 and precision was 0.6398 ± 0.0305 . Thus, only the true-positive subset can carry detected real-anomaly evidence downstream, while the false-negative subset is absent from a deployed gate. The false-positive windows were assigned by Stage 2 to External Position (128.8), Global Position (216.4), Altitude (88.4), and Mechanical/Electrical (98.6) on average. These are predicted suspect categories for false alarms, not validated fault labels. This gated analysis is reported separately from the 1393-window ground-truth-anomaly-conditioned analysis.

Figure 3 summarizes the masking comparison and conservatively filtered commit-level source-tree candidates. Selected frozen analysis records are described in the Data Availability Statement.

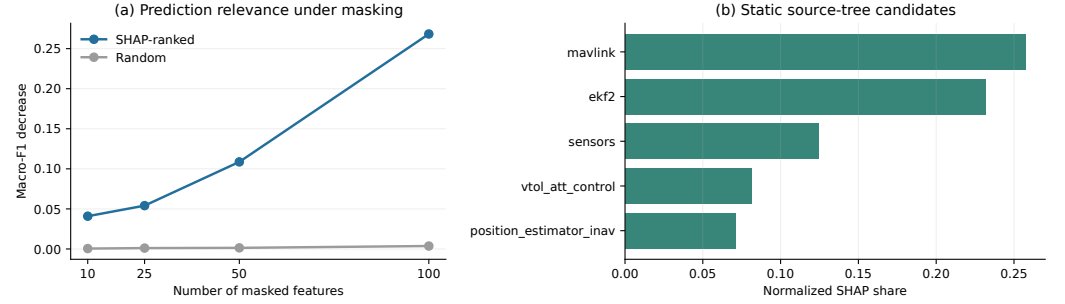


Figure 3. Prediction-relevant evidence and architecture-aware candidates. (a) Mean Macro-F1 decrease after masking validation-ranked SHAP features or equal-size random subsets; uncertainty for paired contrasts is reported in the text, rather than mixing seed variation with random-mask repetitions. (b) Filtered producer-only source-tree candidates from the 1393 ground-truth anomalous test windows whose firmware matches the selected commit; bars are mean normalized shares across seeds. These aggregate real-flight candidates are not mutation-trial localization scores. Complete semantic-map sensitivity is retained in Supplementary Material Section S2.

4.5. Strict Validation, External Validation, and Capability Boundaries

This experiment asks how stricter isolation, external data, and weak labels constrain the evidence established by the primary protocol. The stricter protocols reduced fixed chronological performance and changed the numerical ordering of the complete five-class models. Across five seeds under the 14-window purged protocol, Stage 1 AUPRC was 0.6006 ± 0.0062 , Stage 2 Macro-F1 was 0.7944 ± 0.0049 , Cascade Macro-F1 was 0.5683 ± 0.0177 , and Direct Five-Class Macro-F1 was 0.6024 ± 0.0049 . The paired Direct-minus-Cascade difference was 0.0340, with a flight-log-bootstrap 95% CI of $[-0.0047, 0.0762]$. The mean ordering reversed relative to the fixed chronological split, but the paired interval included zero and therefore did not support a stable purged-protocol advantage. Figure 4 summarizes these protocol-level comparisons.

Leave-log-out evaluation assigned complete logs to the partitions, with 970 logs for training, 206 for validation, and 213 for testing. Across five seeds, Stage 1 AUPRC was 0.6296 ± 0.0023 and AUROC was 0.9265 ± 0.0008 . At the validation-derived operating threshold, Stage 1 F1 was 0.5989 ± 0.0042 and log-aware event F1 was 0.4533 ± 0.0241 . The anomaly-only Stage 2 LightGBM classifier achieved Macro-F1 of 0.9041 ± 0.0029 and Balanced Accuracy of 0.8876 ± 0.0040 . On the complete five-class task, the Cascade achieved Macro-F1 of 0.5962 ± 0.0048 and Balanced Accuracy of 0.5840 ± 0.0057 , while Direct Five-Class achieved Macro-F1 of 0.6092 ± 0.0043 and Balanced Accuracy of 0.6341 ± 0.0022 .

The paired Direct-minus-Cascade Macro-F1 difference under leave-log-out was 0.0130, with a flight-log bootstrap 95% confidence interval of $[-0.0282, 0.0438]$ based on 1000 repetitions. Because the interval includes zero, these results do not support a stable performance difference or a general performance advantage for the hierarchical Cascade. The Cascade remains a distinct gated decision structure, but its fixed-split performance ordering should not be interpreted as a protocol-independent superiority.

The ALFA experiment evaluated zero-shot binary anomaly detection using 16 semantically shared physical channels and a threshold fixed from UAV-SEAD validation data. On 46 ALFA sequences and 5273 ALFA windows, the transferred detector obtained AUPRC of 0.3181 ± 0.0086 (95% sequence-cluster CI $[0.2540, 0.3956]$) and AUROC of 0.6679 ± 0.0069 . The ALFA faults were not mapped to the four UAV-SEAD fault domains; these values therefore assess binary score transfer only. The separate flight-level screening of 139 Uncategorized logs against 898 pure-Normal logs produced flight-level AUPRC of 0.2784 ± 0.0046 (95% flight-cluster CI $[0.2086, 0.3664]$), AUROC of 0.6610 ± 0.0049 , F1 of 0.3307 ± 0.0142 , Uncategorized recall of 0.4118 ± 0.0314 , and Normal specificity of 0.8575 ± 0.0073 . Be-

cause Uncategorized annotations have no temporal ranges, this screening does not support point-wise or event-level claims.

Table 3 separates model-seed standard deviations from sampling uncertainty for the principal absolute metrics. The cluster intervals are wider than the seed dispersions because logs or sequences, rather than highly overlapping windows, are the resampling units.

Table 3. Model and sampling uncertainty for the principal absolute metrics.

Protocol	Metric	Mean	Seed SD	95% cluster CI
Fixed	Stage 1 AUPRC	0.7522	0.0039	[0.6915, 0.8080]
Fixed	Stage 2 Macro-F1	0.8980	0.0020	[0.8581, 0.9296]
Fixed	Cascade Macro-F1	0.6902	0.0071	[0.6308, 0.7350]
Fixed	Direct Five-Class Macro-F1	0.6599	0.0029	[0.5991, 0.7070]
Purged	Stage 1 AUPRC	0.6006	0.0062	[0.4968, 0.7109]
Purged	Stage 2 Macro-F1	0.7944	0.0049	[0.7040, 0.8489]
Purged	Cascade Macro-F1	0.5683	0.0177	[0.4532, 0.6338]
Purged	Direct Five-Class Macro-F1	0.6024	0.0049	[0.4890, 0.6664]
Leave-log-out	Stage 1 AUPRC	0.6296	0.0023	[0.4945, 0.7456]
Leave-log-out	Stage 2 Macro-F1	0.9041	0.0029	[0.8276, 0.9549]
Leave-log-out	Cascade Macro-F1	0.5962	0.0048	[0.4889, 0.6799]
Leave-log-out	Direct Five-Class Macro-F1	0.6092	0.0043	[0.5091, 0.6836]
ALFA zero-shot	Stage 1 AUPRC	0.3181	0.0086	[0.2540, 0.3956]
Uncategorized	Flight-level AUPRC	0.2784	0.0046	[0.2086, 0.3664]

Cluster intervals use 1000 repetitions (seed 20260825). UAV-SEAD resamples complete test logs, ALFA resamples sequences, and Uncategorized screening resamples flights.

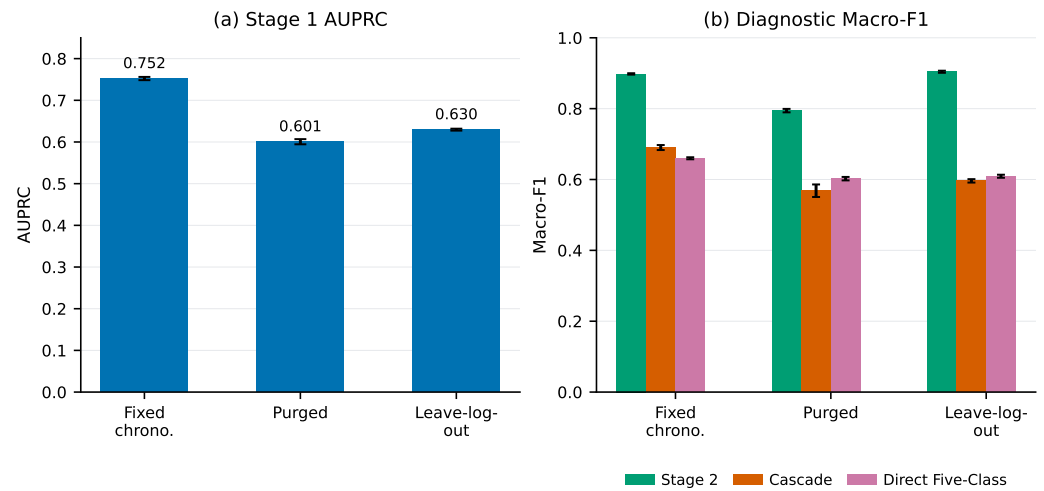


Figure 4. Primary protocol comparison for Stage 1 detection and complete or conditional diagnosis. The fixed chronological, purged, and leave-log-out protocols are evaluated with their respective split definitions. In every panel, error bars show standard deviations across five frozen model seeds, not sampling confidence intervals; absolute flight-log cluster intervals are reported in Table 3. Purging reverses the mean ordering of Cascade and Direct Five-Class Macro-F1, but the paired flight-log interval for their difference includes zero.

4.6. Controlled Paired-Replay Extension

This experiment asks whether a matched healthy replay can provide useful detection-gated module evidence, and whether controlling sampling support improves the distinction between healthy and mutated runs. It evaluates the residual extension in Section 3.7, separately from the learned Stage 1–Stage 2–SHAP chain. In the corrected development comparison, the original learned chain detected 10/12 fault runs but also alarmed on 10/12 independent normal targets; no pair met the clean fault-specific detection criterion. Replacing its gate and then its module evidence with paired residuals yielded 10/12

detections, 1/12 normal alarms, Top-3 coverage of 10/12, and MRR 0.5833. The three-method comparison is provided in Supplementary Material Section S3. These development results motivated a frozen-parameter transfer evaluation rather than a reliability claim.

The main transfer result identifies a useful preprocessing improvement. On the same three transfer source flights, common-support residual computation reduced normal alarms from 11/12 to 3/12 under the frozen paired-residual gate, while retaining 10/12 fault detections. Top-3 and Top-5 both became 10/12, and MRR was 0.6667 (Table 4). The common-support modification was examined after these transfer flights had been inspected and is therefore exploratory. It does not constitute a new independent confirmation, and its benefits cannot be attributed to retraining the original detector or to SHAP.

Table 4. Paired residuals on the same transfer targets: frozen parameters and exploratory common-support processing.

Measure	Frozen paired residual frontend	Common-support frontend
Fault detections	12/12	10/12
Normal runs with an alarm	11/12	3/12
Fault runs with pre-onset alarms	10/12	3/12
Clean fault-specific detections	1/12	7/12
Top-1	6/12	6/12
Top-3	9/12	10/12
Top-5	9/12	10/12
MRR	0.6369	0.6667
Median detection delay (s)	2.023	11.623
Maximum detection delay (s)	54.819	107.623

The threshold, feature noise and publisher allocation are unchanged. Both columns use 12 fault and 12 normal targets from the same three sources; normal alarms cover the complete available run. Delays are conditional on detection (12 and 10 detections, respectively); misses are retained in ranking denominators. The right column was evaluated after transfer data exposure.

The two methods expose different detection trade-offs. The frozen reference method detected all 12 fault runs, but ten had already alarmed before injection and eleven corresponding normal targets alarmed. Common-support processing reduced the number of pre-injection fault alarms to three and increased clean fault-specific detections from one to seven. Its valid channel-run window fraction averaged 83.3% (median 97.6%); unavailable channel windows contributed no evidence. Thus, the comparison includes a change in usable observation support as well as the residual calculation, and coverage accompanies the reported improvement.

The remaining failures are concentrated by mutation. Under common support, Commander and Land Detector each produced 3/3 detections without paired normal alarms. All three EKF2 normal targets alarmed, and all three EKF2 fault runs alarmed before injection, so their 3/3 post-onset recall does not establish fault-specific detection. INAV produced only 1/3 detections, with a delay of 107.623 s; its other two trials remain misses. INAV and Land Detector rankings include tied publishers and do not identify a unique root cause. One Commander transfer trial failed the pre-existing effect-exclusivity check and remains in the 12-trial denominator; its score is not treated as an unambiguous diagnostic success. Supplementary Material Section S4 gives the per-mutation and per-run results.

Channel-specific calibration recovered stronger INAV candidate evidence, but the original-cache variant still produced 8/12 normal alarms despite Top-3/5 of 12/12. Further common-support calibration, within-flight calibration, and a two-healthy-reference variant did not resolve the overall recall-false-alarm trade-off. Their complete comparisons, including 24 additional healthy replays and a control on the same new normal targets, are retained in Supplementary Material Section S5. They are not combined with the best metrics of Table 4 to define a synthetic best method.

5. Discussion

5.1. Principal Findings

For RQ1, the strongest diagnostic result is the learnability of the four annotated fault domains once anomalous windows are supplied. Stage 2 Macro-F1 was 0.8980 under the fixed chronological protocol and 0.9041 under leave-log-out. The complete Cascade is more demanding because Stage 1 can miss anomalies or introduce false alarms; its results and the Direct Five-Class comparison are therefore reported separately. The lower detector AUPRC under strict isolation limits the transfer of fixed-split performance to new flights.

For RQ2, hierarchical evidence preserves the provenance of each statistical contribution and supports a quantitative perturbation test. Validation-ranked SHAP masking caused a greater diagnostic performance decrease than equal-size random masking under both replacement strategies. Conservation verifies the aggregation implementation, while the weaker External and Global Position semantic agreement shows that model relevance and physical fault-domain agreement are different properties.

For RQ3, the commit-specific graph converts topic evidence into source-tree inspection candidates, and the controlled paired-replay extension provides a bounded test against known mutated modules. Common-support residual processing reduced normal alarms from 11/12 to 3/12 while retaining 10/12 detections on the examined transfer set. This positive result coexists with EKF2 false alarms, delayed or missed INAV detections, and static publisher ties. It supports further investigation of diagnostic evidence under matched replay conditions, without establishing reliable end-to-end software fault localization.

5.2. Interpretation of the Hierarchical Model under Strict Protocols

The hierarchy separates two engineering questions that have different label spaces: whether a telemetry window is abnormal and which labeled fault domain best describes an abnormal window. This separation makes the decision path explicit and permits fault-domain-specific evidence to be generated only after an anomaly decision. It also clarifies why Stage 2 performance and complete Cascade performance differ. Stage 2 is evaluated on anomaly-only windows, whereas the Cascade inherits false negatives and false positives from the detector before applying the fault-domain classifier.

The protocol comparisons show that this explicit decision structure should not be conflated with predictive superiority. On the fixed chronological split, the paired Direct-minus-Cascade Macro-F1 contrast was -0.0303 (95% flight-log-bootstrap CI $[-0.0524, -0.0086]$), favoring the Cascade for that split. Purging reversed the mean ordering, but its paired interval included zero; the leave-log-out interval likewise included zero. The available evidence thus does not support a protocol-independent advantage for either complete five-class formulation. The contribution of the hierarchy lies instead in its explicit gating semantics, its separation of detector and diagnostic errors, and its traceable explanation structure.

The degradation from the fixed chronological protocol to stricter isolation also has methodological implications. Overlapping windows from the same flight can share local operating conditions even when assigned chronologically, and a boundary purge removes only part of this proximity. Assigning entire flight logs to disjoint partitions provides a stronger estimate of transfer to unseen flights. The lower leave-log-out Stage 1 AUPRC and event F1 indicate that the detector is the principal bottleneck in the complete chain, whereas the stable anomaly-only Stage 2 result suggests that labeled fault-domain structure remains learnable once abnormal windows are supplied. Future model comparisons should therefore retain flight-log-level isolation and report detector, conditional diagnostic, and complete-task metrics separately.

The resource-limited alternative-model executions remain feasibility references; their objectives and budgets are documented with the complete results in Supplementary Material Section S6. They are not used to claim state-of-the-art superiority.

5.3. *Replay Validity, Specificity, and Module Evidence*

The corrected replay experiments support a limited but concrete positive finding: paired residuals can carry module-candidate evidence, and controlling sampling support can reduce normal alarms. This finding depends on an independent healthy reference for the same source flight and on the fixed PX4 revision. It demonstrates an offline controlled extension, not a repair of the learned Stage 1–Stage 2–SHAP chain.

Execution validity must be separated from detector behavior. An earlier replay attempt stopped updating its input sensors before the scheduled injection; endpoint extension nevertheless generated later feature windows. The historical 0/12 detection result and associated onset-conditioned rankings therefore cannot estimate capability on complete mutation replays. Staging the same input locally, enforcing input coverage, correcting clock alignment, and correcting the generic EKF2 startup mode restored an interpretable comparison. On the corrected development data, the original model detected 10/12 fault runs but alarmed on 10/12 independent normal targets. The unresolved limitation is consequently specificity, rather than an inability to produce any alarm. Supplementary Material Section S1 records the execution audit without using the invalid runs as performance evidence.

The transfer study further shows why high recall or Top- K alone is insufficient. Normal estimator variation and sampling differences can cause sustained residual alarms, while weaker INAV changes may remain below a threshold that suppresses those fluctuations. Common-support processing reduced normal alarms, but missed two INAV trials and detected the third late. Class-specific descriptions explain these observed failures; they were not used as inputs to calibration or ranking. The extra reference experiment also failed to stabilize normal behavior across source flights, so the present data do not justify a reliable cross-flight operating point.

Module evidence remains useful as a candidate representation within these conditions. Uniform publisher allocation can retain the mutated module among a small set, but shared publishers produce ties and the final scores summarize all gated windows offline. Timely detection, unique module identification, and causal fault diagnosis therefore remain distinct requirements. The present results support exploratory paired-replay candidate ranking and a measurable sampling-support improvement, with their false-alarm, coverage, and latency limits retained.

5.4. *Limitations and Future Work*

The real-flight evidence has defined limits. UAV-SEAD labels describe runtime anomalies and fault domains, not faulty source modules. The fixed chronological reproduction permits a source log to span partitions; only the leave-log-out protocol enforces complete-log isolation. The channel dictionary was selected by unlabelled corpus coverage, whereas each protocol's scaler and supervised models use its training data. External validation is binary only, and Uncategorized screening lacks temporal labels.

The explanation and architecture layers are associative. The semantic mapping is author-defined and not independently expert-adjudicated; External and Global Position consistency is weak. The uORB parser's perfect local agreement is an internal shared-vocabulary check, not an independent gold audit. Conservative path exclusion does not reconstruct a target-specific compiled graph. The real-flight architecture analysis covers the 38.7% matching-commit anomalous subset, while propagation depends on topic degree

and publisher/subscriber semantics. Neither this representation nor its masking tests measure engineering inspection time or causal necessity.

The paired-replay extension requires an additional healthy reference from the same source flight and evaluates only four fixed mutation mechanisms at one firmware commit. Its three development and three transfer source flights provide limited independent variation; extra normal replays do not add new source flights. Although replay parameters were frozen before the first transfer evaluation, the historical P2 scaler had already encountered the transfer sources, and subsequent calibration variants used already inspected transfer flights for exploratory assessment. These results are not a fully untouched-pipeline generalization test. Mutation-effect checks failed for one Commander trial in each source set; those trials remain in the primary denominators. Missing or widely separated observations yield unavailable evidence rather than normality. Full-log interpolation and cadence checks, and whole-run evidence aggregation, also preclude an online or real-time claim.

Future work should first isolate healthy EKF2 replay divergence and improve observation-aware residuals without using mutation-specific prediction rules. Any revised method requires separate development and calibration flights followed by a fully untouched source-flight evaluation, including the scaler. Independent semantic adjudication, per-commit graphs, and runtime topic provenance would address the separate limitations of static module evidence. These are extensions of the present evidence framework rather than prerequisites for interpreting its recorded-data classification and explanation results.

6. Conclusions

6.1. Summary and Outlook

HS-AeroTS connects recorded PX4 anomaly detection and fault-domain diagnosis with traceable telemetry evidence and commit-specific software inspection candidates. The conditional four-domain classifier achieved Macro-F1 of 0.8980 ± 0.0020 under the fixed chronological protocol and 0.9041 ± 0.0029 under leave-log-out. These results establish conditional diagnostic performance; the complete five-class task additionally inherits the detector's errors and does not show a stable Cascade advantage under strict isolation.

The evidence layer provides a second supported result. Feature provenance allows TreeSHAP magnitudes to be conserved across aggregation levels, and attribution-guided masking causes greater degradation than random masking under the evaluated perturbations. The static uORB graph expresses matching-commit topic evidence as module candidates, with semantic-map sensitivity, graph coverage, and propagation assumptions explicitly retained.

In an exploratory paired-replay extension, common-support residual computation reduced normal alarms from 11/12 to 3/12 while detecting 10/12 fault trials; Top-3/5 were 10/12 and MRR was 0.6667. This improvement is specific to the examined matched, offline replay setting. Persistent normal EKF2 alarms, delayed or missed INAV detections, and limited source-flight independence prevent a reliable cross-flight module-diagnosis claim. The resulting contribution is a reproducible diagnostic evidence framework with demonstrated classification and perturbation-test results, together with a controlled account of where module-candidate ranking remains limited.

Supplementary Materials: Supplementary Material: corrected replay validity and source roles; semantic-map sensitivity; corrected development comparisons; transfer per-mutation and per-run results; complete calibration and reference ablations; resource-limited baselines and reproducibility details.

Author Contributions: Conceptualization, Jiao Wu, Chenglie Du and Junsheng Wu; methodology, Jiao Wu and Bo Liu; software, Jiao Wu and Bo Liu; validation, Jiao Wu, Bo Liu and Yilan Li; formal analysis, Jiao Wu, Bo Liu and Yilan Li; investigation, Jiao Wu; data curation, Jiao Wu; writing—

original draft preparation, Jiao Wu; writing—review and editing, all authors; visualization, Jiao Wu; supervision, Chenglie Du and Junsheng Wu; project administration, Chenglie Du and Junsheng Wu. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The UAV-SEAD data used for the primary experiments and weak-label screening are publicly available from Hugging Face at <https://huggingface.co/datasets/aykutkabaoglu/uav-flight-anomaly-dataset> under the Creative Commons Attribution 4.0 International license; the analyses used repository revision 774a336a1d1d06b248927a35c2ea1f6869e9e1a4. The processed ALFA data used for zero-shot external validation are publicly available from Figshare at <https://figshare.com/articles/dataset/12707963> (article 12707963, file 24095870) under the Creative Commons Attribution 4.0 International license. The controlled replay experiments used PX4-Autopilot source code at commit 82aa24adfca29321cfd1209e287eab6c2b16780e, available at <https://github.com/PX4/PX4-Autopilot/commit/82aa24adfca29321cfd1209e287eab6c2b16780e>. No new primary flight data were collected. The accompanying Supplementary Material reports corrected replay protocols, complete auxiliary comparisons, and per-trial outcomes. The public analysis repository at <https://github.com/lbexplorer/HS-AeroTS>, checked on 8 September 2026 at commit 264d1f70ef78bba19d56eed1b8b9fd679056db84, contains selected code and corrected P9 summaries. The subsequent common-support (P15) artifacts and this manuscript revision were not present at that public commit; their result tables and source manifest accompany the revision package. Raw replay outputs and trained model files are not implied to be included by this repository reference. The article and appendices provide the complete semantic mapping, software versions, exclusion rules, split counts, and statistical settings. Requests for additional artifacts remain subject to the underlying data and software licenses.

Acknowledgments: During the preparation of this manuscript and study, the authors used OpenAI Codex during August–September 2026 to assist with analysis and replay-experiment code, result interpretation, manuscript revision, and figure preparation. The authors executed and checked the analyses, reviewed and edited the output, and take full responsibility for the content of this publication. No individuals or organizations are otherwise acknowledged.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Document-Derived Semantic Channel Mapping

Table A1 gives the complete mapping used for the semantic agreement sensitivity. It was defined from PX4 topic and field semantics without access to SHAP values. “Shared” is a contextual or cross-domain assignment and is counted as a non-match for every specific fault-domain label. Each channel contributes 18 temporal–statistical features.

Table A1. Complete 87-channel document-derived semantic mapping.

Telemetry channel	PX4 topic	Declared subsystem
actuator_controls_0.control[0]	actuator_controls_0	Mechanical/Electrical
actuator_controls_0.control[1]	actuator_controls_0	Mechanical/Electrical
actuator_controls_0.control[2]	actuator_controls_0	Mechanical/Electrical
actuator_controls_0.control[3]	actuator_controls_0	Mechanical/Electrical
actuator_outputs.output[0]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[1]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[2]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[3]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[4]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[5]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[6]	actuator_outputs	Mechanical/Electrical
actuator_outputs.output[7]	actuator_outputs	Mechanical/Electrical

Continued on next page

Table A1 – continued from previous page

Telemetry channel	PX4 topic	Declared subsystem
battery_status.current_a	battery_status	Mechanical/Electrical
battery_status.current_filtered_a	battery_status	Mechanical/Electrical
battery_status.discharged_mah	battery_status	Mechanical/Electrical
battery_status.remaining	battery_status	Mechanical/Electrical
battery_status.voltage_filtered_v	battery_status	Mechanical/Electrical
battery_status.voltage_v	battery_status	Mechanical/Electrical
cpuload.load	cpuload	Mechanical/Electrical
cpuload.ram_usage	cpuload	Mechanical/Electrical
distance_sensor.current_distance	distance_sensor	Altitude
ekf2_innovations.hagl_innov	ekf2_innovations	Altitude
ekf2_innovations.heading_innov	ekf2_innovations	Global Position
ekf2_innovations.output_tracking_error[0]	ekf2_innovations	Shared
ekf2_innovations.output_tracking_error[1]	ekf2_innovations	Shared
ekf2_innovations.output_tracking_error[2]	ekf2_innovations	Shared
ekf2_innovations.vel_pos_innov[3]	ekf2_innovations	Global Position
ekf2_innovations.vel_pos_innov[4]	ekf2_innovations	Global Position
ekf2_innovations.vel_pos_innov[5]	ekf2_innovations	Altitude
estimator_status.hagl_test_ratio	estimator_status	Altitude
estimator_status.hgt_test_ratio	estimator_status	Altitude
estimator_status.mag_test_ratio	estimator_status	Shared
estimator_status.pos_horiz_accuracy	estimator_status	Global Position
estimator_status.pos_test_ratio	estimator_status	Global Position
estimator_status.pos_vert_accuracy	estimator_status	Altitude
estimator_status.vibe[0]	estimator_status	Mechanical/Electrical
estimator_status.vibe[1]	estimator_status	Mechanical/Electrical
estimator_status.vibe[2]	estimator_status	Mechanical/Electrical
manual_control_setpoint.r	manual_control_setpoint	Shared
manual_control_setpoint.x	manual_control_setpoint	Shared
manual_control_setpoint.y	manual_control_setpoint	Shared
manual_control_setpoint.z	manual_control_setpoint	Shared
rate_ctrl_status.pitchspeed_integ	rate_ctrl_status	Mechanical/Electrical
rate_ctrl_status.rollspeed_integ	rate_ctrl_status	Mechanical/Electrical
rate_ctrl_status.yawspeed_integ	rate_ctrl_status	Mechanical/Electrical
sensor_combined.accelerometer_m_s2[0]	sensor_combined	Mechanical/Electrical
sensor_combined.accelerometer_m_s2[1]	sensor_combined	Mechanical/Electrical
sensor_combined.accelerometer_m_s2[2]	sensor_combined	Mechanical/Electrical
sensor_combined.gyro_rad[0]	sensor_combined	Mechanical/Electrical
sensor_combined.gyro_rad[1]	sensor_combined	Mechanical/Electrical
sensor_combined.gyro_rad[2]	sensor_combined	Mechanical/Electrical
sensor_preflight.accel_inconsistency_m_s_s	sensor_preflight	Mechanical/Electrical
sensor_preflight.gyro_inconsistency_rad_s	sensor_preflight	Mechanical/Electrical
vehicle_attitude.pitchspeed	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[0]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[1]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[2]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.q[3]	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.rollspeed	vehicle_attitude	Mechanical/Electrical
vehicle_attitude.yawspeed	vehicle_attitude	Mechanical/Electrical
vehicle_attitude_setpoint.pitch_body	vehicle_attitude_setpoint	Mechanical/Electrical
vehicle_attitude_setpoint.roll_body	vehicle_attitude_setpoint	Mechanical/Electrical
vehicle_attitude_setpoint.yaw_body	vehicle_attitude_setpoint	Mechanical/Electrical
vehicle_land_detected.ground_contact	vehicle_land_detected	Altitude
vehicle_land_detected.landed	vehicle_land_detected	Altitude
vehicle_land_detected.maybe_landed	vehicle_land_detected	Altitude
vehicle_local_position.ax	vehicle_local_position	Global Position
vehicle_local_position.ay	vehicle_local_position	Global Position
vehicle_local_position.az	vehicle_local_position	Altitude
vehicle_local_position.dist_bottom	vehicle_local_position	Altitude
vehicle_local_position.dist_bottom_rate	vehicle_local_position	Altitude
vehicle_local_position.eph	vehicle_local_position	Global Position
vehicle_local_position.epv	vehicle_local_position	Altitude
vehicle_local_position.vx	vehicle_local_position	Global Position
vehicle_local_position.vy	vehicle_local_position	Global Position
vehicle_local_position.vz	vehicle_local_position	Altitude
vehicle_local_position.x	vehicle_local_position	Global Position
vehicle_local_position.y	vehicle_local_position	Global Position
vehicle_local_position.z	vehicle_local_position	Altitude
vehicle_rates_setpoint.pitch	vehicle_rates_setpoint	Mechanical/Electrical

Continued on next page

Table A1 – continued from previous page

Telemetry channel	PX4 topic	Declared subsystem
vehicle_rates_setpoint.roll	vehicle_rates_setpoint	Mechanical/Electrical
vehicle_rates_setpoint.yaw	vehicle_rates_setpoint	Mechanical/Electrical
vehicle_status.arming_state	vehicle_status	Shared
vehicle_status.nav_state	vehicle_status	Shared
vehicle_vision_position.x	vehicle_vision_position	External Position
vehicle_vision_position.y	vehicle_vision_position	External Position
vehicle_vision_position.z	vehicle_vision_position	External Position

Appendix B. Reproducibility and Statistical Details

Appendix B.1. Software and Frozen Versions

The earlier frozen-prediction recomputation (P13) used Python 3.12.13, NumPy 2.3.5, pandas 3.0.1, scikit-learn 1.8.0, LightGBM 4.6.0, joblib 1.5.2, PyYAML 6.0.3, and pyulog 1.2.4. TreeSHAP contributions were obtained from LightGBM's `pred_contrib` interface. No supervised model was retrained or tuned for this revision. The separate replay residual and calibration variants are described in Section 3.7 and the Supplementary Material; they do not alter the real-flight models. Model seeds were 0–4. The PX4 source-tree scan and controlled replay used commit 82aa24adfca29321cfd1209e287eab6c2b16780e. The CATCH feasibility execution used the official core at commit 3647c69be5eb56649b072596cf89098e689e20c3.

Appendix B.2. Bootstrap and Split Settings

Absolute real-flight confidence intervals used 1000 complete-flight cluster-bootstrap repetitions with random seed 20260825. Each bootstrap sample was evaluated for all five frozen model seeds and the replicate metric was their mean. ALFA used the same repetition count and seed with its 46 sequences as clusters; Uncategorized screening used its 1017 flights. Historical replay intervals used incomplete input runs and are excluded from the revised capability results. Corrected replay counts and their source-flight identities are provided descriptively in the Supplementary Material; no historical interval is reused. The fixed chronological split contained 152,340/32,138/34,059 train/validation/test windows; the 14-window purged split contained 133,353/8,536/17,966; leave-log-out contained 970/206/213 train/validation/test logs and 149,774/37,324/31,439 windows. All window models used length 96, stride 8, prediction horizon 12, 87 channels, and 1566 derived features.

Appendix B.3. Conservative Source-Path Exclusion

The confirmed firmware metadata indicated PX4FMU_V2 with NuttX. The conservative sensitivity therefore excluded edges whose normalized source path contained markers for examples, templates, tests, POSIX, QURT, Bebop, Linux, Snapdragon, simulator, or replay code. The rule removed 35 of 180 edges: 14 for examples, 8 for POSIX, 7 for simulator/replay, 2 for QURT, and one each for Linux, Snapdragon, tests, and templates. The remaining 145 edges covered 36 modules and all 18 evaluated topics. The filtered table is a strict subset of the original table and passed the same evidence-conservation calculation. This path-based sensitivity is not equivalent to resolving CMake configuration, board manifests, conditional compilation, linking, or runtime activation for an actual PX4FMU_V2 build.

References

1. Fourlas, G.K.; Karras, G.C. A Survey on Fault Diagnosis and Fault-Tolerant Control Methods for Unmanned Aerial Vehicles. *Machines* **2021**, *9*, 197. <https://doi.org/10.3390/machines9090197>.

2. Puchalski, R.; Giernacki, W. UAV Fault Detection Methods, State-of-the-Art. *Drones* **2022**, *6*, 330. <https://doi.org/10.3390/drones6110330>.
3. Tan, I.W.H.; Poskitt, C.M.; Jiang, L.; Shar, L.K. Anomaly Management in Unmanned Aerial Vehicles: A Systematic Literature Review. *IEEE Access* **2026**, *14*, 117991–118010. <https://doi.org/10.1109/ACCESS.2026.3716292>.
4. Keipour, A.; Mousaei, M.; Scherer, S. Automatic Real-Time Anomaly Detection for Autonomous Aerial Vehicles. In Proceedings of the 2019 International Conference on Robotics and Automation (ICRA), Montreal, QC, Canada, 20–24 May 2019; pp. 5679–5685. <https://doi.org/10.1109/ICRA.2019.8794286>.
5. Hu, G.; Zhou, Z.; Li, Z.; Dong, Z.; Fang, J.; Zhao, Y.; Zhou, C. Multiscale Transformers With Contrastive Learning for UAV Anomaly Detection. *IEEE Trans. Instrum. Meas.* **2025**, *74*, 1–15. <https://doi.org/10.1109/TIM.2025.3571126>.
6. Kabaoglu, A.; Sariel, S. UAV-SEAD: State Estimation Anomaly Dataset for UAVs. *arXiv* **2026**, arXiv:2602.13900. <https://arxiv.org/abs/2602.13900>.
7. Yang, J.; Zhang, Y.; Wang, J.; Miao, Q. A Dynamics-Constrained Probabilistic Method for Adaptive Anomaly Detection in UAVs. *IEEE Trans. Instrum. Meas.* **2026**, *75*, 3521111. <https://doi.org/10.1109/TIM.2026.3718566>.
8. Lundberg, S.M.; Lee, S.-I. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
9. Lundberg, S.M.; Erion, G.; Chen, H.; DeGrave, A.; Prutkin, J.M.; Nair, B.; Katz, R.; Himmelfarb, J.; Bansal, N.; Lee, S.-I. From Local Explanations to Global Understanding with Explainable AI for Trees. *Nat. Mach. Intell.* **2020**, *2*, 56–67. <https://doi.org/10.1038/s42256-019-0138-9>.
10. Wong, W.E.; Gao, R.; Li, Y.; Abreu, R.; Wotawa, F. A Survey on Software Fault Localization. *IEEE Trans. Softw. Eng.* **2016**, *42*, 707–740. <https://doi.org/10.1109/TSE.2016.2521368>.
11. Meier, L.; Honegger, D.; Pollefeys, M. PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms. In Proceedings of the 2015 IEEE International Conference on Robotics and Automation (ICRA), Seattle, WA, USA, 26–30 May 2015; pp. 6235–6240. <https://doi.org/10.1109/ICRA.2015.7140074>.
12. PX4 Development Team. uORB Messaging and Publication/Subscription Graph. Available online: <https://docs.px4.io/main/en/middleware/uorb> (accessed on 25 August 2026).
13. Pearson, S.; Campos, J.; Just, R.; Fraser, G.; Abreu, R.; Ernst, M.D.; Pang, D.; Keller, B. Evaluating and Improving Fault Localization. In Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering (ICSE), Buenos Aires, Argentina, 20–28 May 2017; pp. 609–620. <https://doi.org/10.1109/ICSE.2017.62>.
14. Papadakis, M.; Kintis, M.; Zhang, J.; Jia, Y.; Le Traon, Y.; Harman, M. Mutation Testing Advances: An Analysis and Survey. *Adv. Comput.* **2019**, *112*, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>.
15. Adaika, Z.; Al-Haddad, L.A.; Giernacki, W.; Jaber, A.A.; Boumehraz, M.; Hamzah, M.N.; Flayyih, M.A. Fault Detection and Diagnosis Methodologies for Unmanned Aerial Vehicles: State-of-the-Art. *J. Intell. Robot. Syst.* **2025**, *111*, 63. <https://doi.org/10.1007/s10846-025-02267-8>.
16. Keipour, A.; Mousaei, M.; Scherer, S. ALFA: A Dataset for UAV Fault and Anomaly Detection. *Int. J. Robot. Res.* **2021**, *40*, 515–520. <https://doi.org/10.1177/0278364920966642>.
17. Liu, F.T.; Ting, K.M.; Zhou, Z.-H. Isolation Forest. In Proceedings of the 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 15–19 December 2008; pp. 413–422. <https://doi.org/10.1109/ICDM.2008.17>.
18. Zhang, C.; Song, D.; Chen, Y.; Feng, X.; Lumezanu, C.; Cheng, W.; Ni, J.; Zong, B.; Chen, H.; Chawla, N.V. A Deep Neural Network for Unsupervised Anomaly Detection and Diagnosis in Multivariate Time Series Data. *Proc. AAAI Conf. Artif. Intell.* **2019**, *33*, 1409–1416. <https://doi.org/10.1609/aaai.v33i01.33011409>.
19. Su, Y.; Zhao, Y.; Niu, C.; Liu, R.; Sun, W.; Pei, D. Robust Anomaly Detection for Multivariate Time Series through Stochastic Recurrent Neural Network. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 2828–2837. <https://doi.org/10.1145/3292500.3330672>.

20. Deng, A.; Hooi, B. Graph Neural Network-Based Anomaly Detection in Multivariate Time Series. *Proc. AAAI Conf. Artif. Intell.* **2021**, *35*, 4027–4035. <https://doi.org/10.1609/aaai.v35i5.16523>.
21. Tuli, S.; Casale, G.; Jennings, N.R. TranAD: Deep Transformer Networks for Anomaly Detection in Multivariate Time Series Data. *Proc. VLDB Endow.* **2022**, *15*, 1201–1214. <https://doi.org/10.14778/3514061.3514067>.
22. Xu, J.; Wu, H.; Wang, J.; Long, M. Anomaly Transformer: Time Series Anomaly Detection with Association Discrepancy. In Proceedings of the International Conference on Learning Representations, Virtual, 25–29 April 2022. Available online: https://openreview.net/forum?id=LzQQ89U1qm_ (accessed on 25 August 2026).
23. Wu, X.; Qiu, X.; Li, Z.; Wang, Y.; Hu, J.; Guo, C.; Xiong, H.; Yang, B. CATCH: Channel-Aware Multivariate Time Series Anomaly Detection via Frequency Patching. In Proceedings of the International Conference on Learning Representations, Singapore, 24–28 April 2025. Available online: <https://openreview.net/forum?id=m08aK3xxdJ> (accessed on 25 August 2026).
24. Liu, Z.; Gao, M.; Jiao, P. GCAD: Anomaly Detection in Multivariate Time Series from the Perspective of Granger Causality. *Proc. AAAI Conf. Artif. Intell.* **2025**, *39*, 19041–19049. <https://doi.org/10.1609/aaai.v39i18.34096>.
25. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.-Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2017; Volume 30.
26. Wei, J.; Li, H.; Li, Y.; Zhao, Y.; Yao, D.; Lu, B.; Ye, X.; Im, S.-K.; Wang, Y.; Yang, X. AeroTSBoost: Temporal-Statistical Boosting for Real-World UAV Telemetry Anomaly Mining. *arXiv* **2026**, arXiv:2605.25639. <https://arxiv.org/abs/2605.25639>.
27. Davis, J.; Goadrich, M. The Relationship between Precision-Recall and ROC Curves. In Proceedings of the 23rd International Conference on Machine Learning, Pittsburgh, PA, USA, 25–29 June 2006; pp. 233–240. <https://doi.org/10.1145/1143844.1143874>.
28. Saito, T.; Rehmsmeier, M. The Precision–Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE* **2015**, *10*, e0118432. <https://doi.org/10.1371/journal.pone.0118432>.
29. Tatbul, N.; Lee, T.J.; Zdonik, S.; Alam, M.; Gottschlich, J. Precision and Recall for Time Series. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2018; Volume 31.
30. Wenig, P.; Schmidl, S.; Papenbrock, T. TimeEval: A Benchmarking Toolkit for Time Series Anomaly Detection Algorithms. *Proc. VLDB Endow.* **2022**, *15*, 3678–3681. <https://doi.org/10.14778/3554821.3554873>.
31. Wu, R.; Keogh, E.J. Current Time Series Anomaly Detection Benchmarks Are Flawed and Are Creating the Illusion of Progress. *IEEE Trans. Knowl. Data Eng.* **2023**, *35*, 2421–2429. <https://doi.org/10.1109/TKDE.2021.3112126>.
32. Ribeiro, M.T.; Singh, S.; Guestrin, C. Why Should I Trust You?: Explaining the Predictions of Any Classifier. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 1135–1144. <https://doi.org/10.1145/2939672.2939778>.
33. Sundararajan, M.; Taly, A.; Yan, Q. Axiomatic Attribution for Deep Networks. In Proceedings of the 34th International Conference on Machine Learning, Sydney, Australia, 6–11 August 2017; Volume 70, pp. 3319–3328.
34. Bento, J.; Saleiro, P.; Cruz, A.F.; Figueiredo, M.A.T.; Bizarro, P. TimeSHAP: Explaining Recurrent Models through Sequence Perturbations. In Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, Virtual, 14–18 August 2021; pp. 2565–2573. <https://doi.org/10.1145/3447548.3467166>.
35. Adebayo, J.; Gilmer, J.; Muelly, M.; Goodfellow, I.; Hardt, M.; Kim, B. Sanity Checks for Saliency Maps. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2018; Volume 31.
36. Hooker, S.; Erhan, D.; Kindermans, P.-J.; Kim, B. A Benchmark for Interpretability Methods in Deep Neural Networks. In *Advances in Neural Information Processing Systems*; Curran Associates, Inc.: Red Hook, NY, USA, 2019; Volume 32.

37. Hedström, A.; Weber, L.; Krakowczyk, D.; Bareeva, D.; Motzkus, F.; Samek, W.; Lapuschkin, S.; Höhne, M.M.-C. Quantus: An Explainable AI Toolkit for Responsible Evaluation of Neural Network Explanations and Beyond. *J. Mach. Learn. Res.* **2023**, *24*, 1–11.
38. Jones, J.A.; Harrold, M.J.; Stasko, J.T. Visualization of Test Information to Assist Fault Localization. In Proceedings of the 24th International Conference on Software Engineering (ICSE 2002), Orlando, FL, USA, 19–25 May 2002; pp. 467–477. <https://doi.org/10.1145/581339.581397>.